

Missionaries and Cannibals Problem

1. Aim

To implement a search algorithm to solve the **Missionaries and Cannibals problem** by safely transporting all missionaries and cannibals from one side of a river to the other without allowing the cannibals to outnumber the missionaries.

2. Problem Statement

The Missionaries and Cannibals problem involves **three missionaries and three cannibals** who need to cross a river using a boat that can carry a maximum of two people at a time. The main challenge is to find a sequence of crossings where the number of cannibals never exceeds the number of missionaries on either side, whenever missionaries are present.

3. Solution

The problem can be solved using a **state-space search approach**, where each state represents the number of missionaries, cannibals, and the position of the boat on each side of the river. Valid states are generated by moving one or two people using the boat, while invalid states are rejected. The search continues until all missionaries and cannibals reach the opposite side safely.

4. Algorithm

1. Start with **3 missionaries and 3 cannibals** on the left side and the boat on the left.
2. Represent the current situation as a state.
3. Generate all possible combinations of one or two people who can travel in the boat.
4. Move the selected people to the opposite side.
5. Check whether the new state is valid.
6. A state is valid only when missionaries are not outnumbered by cannibals on either side, whenever missionaries are present.
7. Avoid states that have already been visited.
8. Continue generating and checking states until all missionaries and cannibals reach the right side.
9. Store the path of states to obtain the sequence of boat movements.

10.Display the solution path.

5. Program

```
from collections import deque
```

```
def is_valid(state):
```

```
    m_left, c_left, boat = state
```

```
    m_right = 3 - m_left
```

```
    c_right = 3 - c_left
```

```
    # Check left side
```

```
    if m_left > 0 and c_left > m_left:
```

```
        return False
```

```
    # Check right side
```

```
    if m_right > 0 and c_right > m_right:
```

```
        return False
```

```
    return True
```

```
def get_next_states(state):
```

```
    m_left, c_left, boat = state
```

```
    moves = [
```

```
        (1, 0), # 1 missionary
```

```
        (2, 0), # 2 missionaries
```

```
        (0, 1), # 1 cannibal
```

```

(0, 2), # 2 cannibals
(1, 1) # 1 missionary and 1 cannibal
]

next_states = []

for m, c in moves:

    if boat == 0: # Boat on left side
        if m <= m_left and c <= c_left:
            new_state = (m_left - m, c_left - c, 1)

            if is_valid(new_state):
                next_states.append((new_state, (m, c)))

    else: # Boat on right side
        m_right = 3 - m_left
        c_right = 3 - c_left

        if m <= m_right and c <= c_right:
            new_state = (m_left + m, c_left + c, 0)

            if is_valid(new_state):
                next_states.append((new_state, (m, c)))

return next_states

```

```
def solve():
    start = (3, 3, 0)
    goal = (0, 0, 1)

    queue = deque([(start, [])])
    visited = {start}

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        for next_state, move in get_next_states(state):
            if next_state not in visited:
                visited.add(next_state)
                queue.append(
                    (next_state, path + [state])
                )

    return None

solution = solve()

print("Solution Path:")
```

for state in solution:

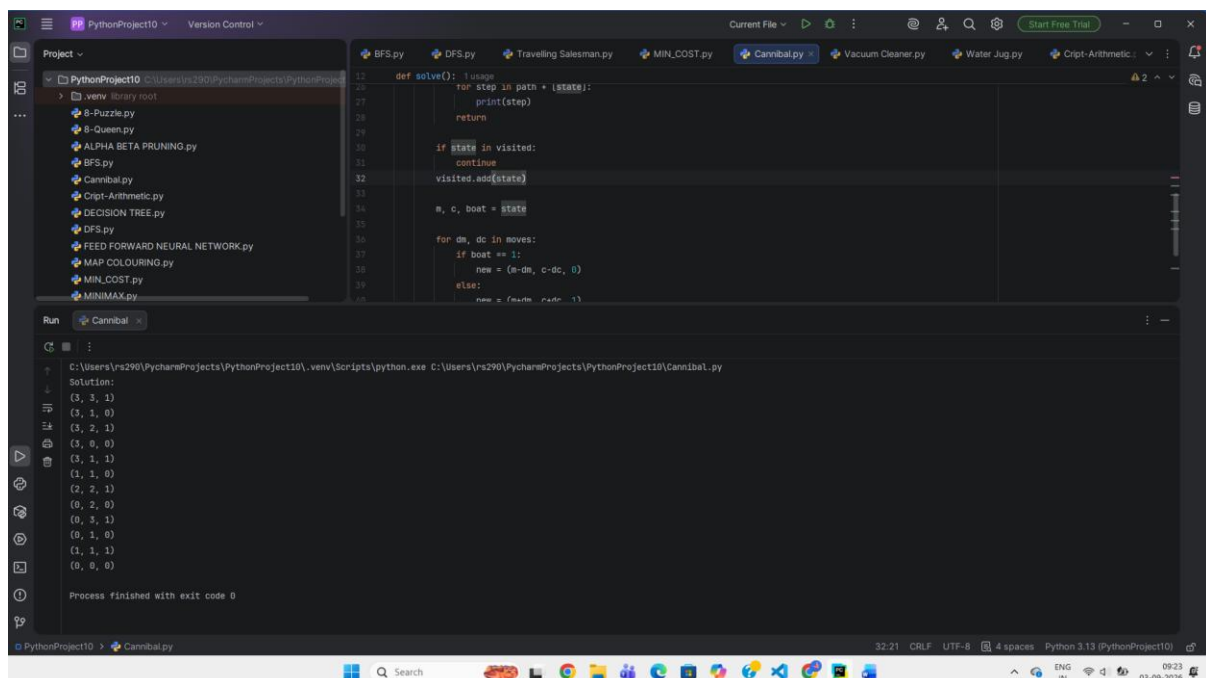
```
m_left, c_left, boat = state
```

```
side = "Left" if boat == 0 else "Right"
```

```
print(  
    f'Missionaries Left: {m_left}, "  
    f'Cannibals Left: {c_left}, "  
    f'Boat: {side}"  
)
```

6. Result

The Missionaries and Cannibals problem was successfully solved using a **state-space search approach**. The program generates valid states and avoids unsafe situations where cannibals outnumber missionaries. It successfully finds a sequence of moves that transfers all **3 missionaries and 3 cannibals** from the left side to the right side of the river safely.



The screenshot shows a Python IDE with a project named 'PythonProject10'. The file explorer on the left lists various Python files, including 'Cannibal.py'. The main editor window displays the code for 'Cannibal.py', which includes a 'solve()' function. The function iterates through a list of states, printing each state and checking if it is in the 'visited' set. The states are represented as tuples (m, c, boat), where m is the number of missionaries, c is the number of cannibals, and boat is the side of the river (0 for Left, 1 for Right). The states are: (3, 3, 1), (3, 1, 0), (3, 2, 1), (3, 0, 0), (3, 1, 1), (1, 1, 0), (2, 2, 1), (0, 2, 0), (0, 3, 1), (0, 1, 0), (1, 1, 1), and (0, 0, 0). The process finished with exit code 0.

Result:

All missionaries and cannibals successfully cross the river while satisfying the problem constraints.